Tenth Implementation Iteration: Space Allocation

with Required Building Constraint

Software Design / Iteration Report

Name: Najib Attar
Date: March 2026

1. Iteration Goal

The goal of this tenth implementation iteration is to extend the prototype by adding a simple building-based location constraint. In previous iterations, the system evaluated requests according to participant count, required features, space availability status, and day/time availability. However, it could not represent the case where a request requires a space in a specific building.

In this iteration, each space is assigned a building value, and each request may optionally include a required building. A request is approved only if the requested space satisfies the capacity requirement, the required feature requirement, the availability status requirement, the building requirement, and the day/time availability rule.

2. Changes from Iteration 9

Compared with the ninth iteration, the following changes were introduced:

- Extended the `Space` model by adding a building field
- Extended `OneTimeRequest` by adding a required building field
- Extended `RecurringRequest` by adding a required building field
- Added a new decision rule: `LocationRule`
- Updated the request-processing logic so that a request is rejected if the requested space does not match the required building
- Preserved the same request types:
  - `OneTimeRequest`
  - `RecurringRequest`

- Preserved the same supported space types:
  - `Classroom`
  - `Laboratory`
  - `MeetingRoom`
- Preserved the same rules:
  - `AvailabilityRule`
  - `CapacityRule`
  - `FeatureRule`
  - `StatusRule`

3. Scenario

A user submits either a one-time request or a recurring request for a specific space resource. The request may include a required feature and a required building.

The system checks whether the requested space matches the required building, whether the required feature is supported, whether the space is currently available, whether the participant count fits the capacity, and whether the requested time slot or all requested occurrences are available.

A one-time request is approved if its single occurrence passes all rules. A recurring request is approved only if all occurrences pass all rules. Otherwise, the request is rejected with a stored rejection reason.

4. Modules and Their Tasks

A. Request Management Module

- Receive one-time or recurring space requests from the user
- Create and store `OneTimeRequest` or `RecurringRequest`
- Store request details including required feature, required building, and rejection reason when applicable
- Send the request to the Rule Evaluation Module

B. Resource Management Module

- Manage all supported space types
- Provide information for `Classroom`, `Laboratory`, and `MeetingRoom`
- Provide space identity, type, name, capacity, supported features, availability status, and building information

C. Rule Evaluation Module

- Apply `CapacityRule`
- Apply `FeatureRule`
- Apply `StatusRule`
- Apply `LocationRule`
- Apply `AvailabilityRule`
- Store a short rejection reason when one of the rules fails
- Return the final decision: Approved or Rejected

D. Allocation Management Module

- Create one allocation for a one-time request if all rules pass
- Create multiple allocations for a recurring request if all occurrences pass
- Update request status to Approved or Rejected

5. Iteration Comparison

Iteration	**Request Types**	**Resource Types**	**Rules Implemented**
Iteration 1	OneTimeRequest	Classroom	AvailabilityRule
Iteration 2	OneTimeRequest	Classroom	AvailabilityRule + CapacityRule
Iteration 3	OneTimeRequest	Classroom + Laboratory	AvailabilityRule + CapacityRule
Iteration 4	OneTimeRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule
Iteration 5	OneTimeRequest + RecurringRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule
Iteration 6	OneTimeRequest + RecurringRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule
Iteration 7	OneTimeRequest + RecurringRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule + FeatureRule
Iteration 8	OneTimeRequest + RecurringRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule + FeatureRule + StatusRule
Iteration 9	OneTimeRequest + RecurringRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule + FeatureRule + StatusRule
Iteration 10	OneTimeRequest + RecurringRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule + FeatureRule + StatusRule + LocationRule

6. Updated Class Diagram

Figure 1 shows the updated simplified class diagram for the tenth implementation iteration.

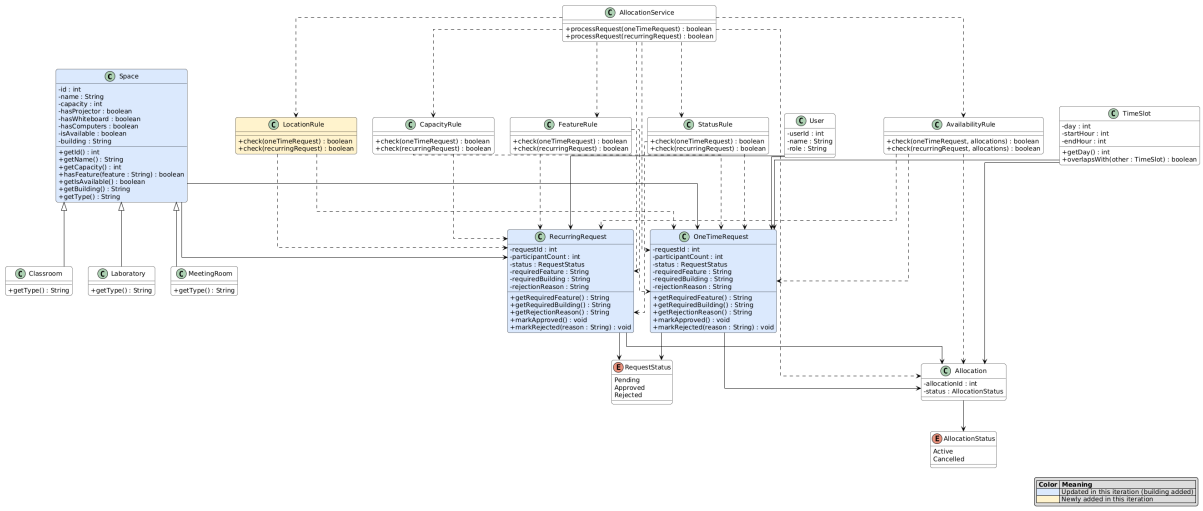

Figure 1: Updated Simplified Class Diagram

7. Updated Workflow Diagram

Figure 2 shows the workflow for the tenth iteration of the space allocation process.

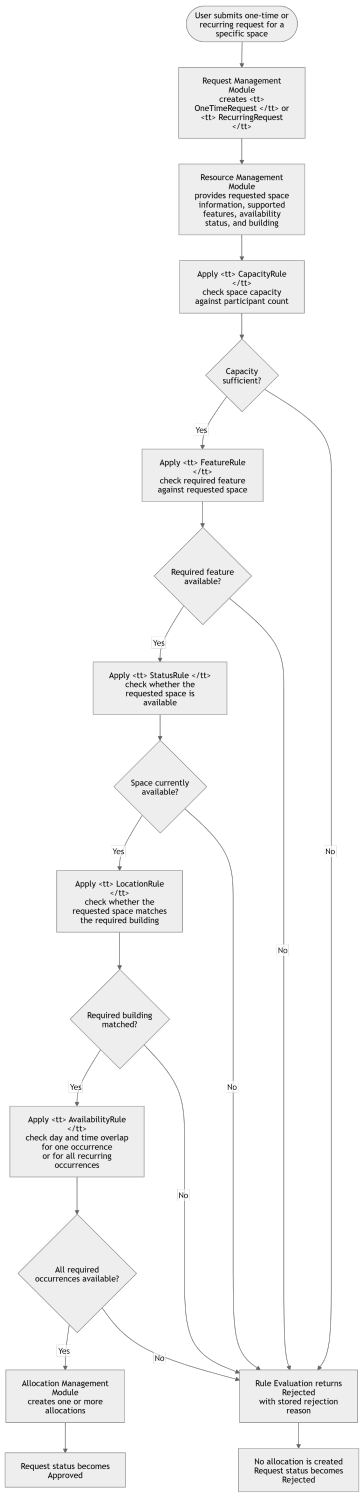

Figure 2: Updated Workflow Diagram

8. Prototype Implementation

To validate the design of this iteration, the C++ prototype was extended by adding a building field to the `Space` model and a required building field to both `OneTimeRequest`

and `RecurringRequest`. A new decision rule, `LocationRule`, was introduced to verify that the requested space matches the required building.

The allocation service continues to support both one-time and recurring requests. It now checks the capacity rule, the feature rule, the status rule, the location rule, and the availability rule before creating allocations.

If all rules pass, one or more allocations are created depending on the request type. If one or more rules fail, the request is rejected and the rejection reason is displayed in the output.

9. Summary

This tenth implementation iteration extends the prototype by introducing a simple building-based location constraint while preserving the existing request types, space types, and allocation architecture. The system can now distinguish spaces not only by type, capacity, features, and availability, but also by building location.

This iteration demonstrates controlled iterative development by adding another small request constraint directly aligned with the domain model. Future iterations can continue this approach by refining location handling or extending request constraints further.